

CSEN 240: Utilizing SCU's WAVE HPC via SLURM

Class	CSEN 240
Professor	Sid Nath
University	Santa Clara University
Date	Feb 2, 2025
Description	<i>Step-by-step instructions on how to connect to the WAVE HPC via your personal computer and run jobs with SLURM.</i>

Table of Contents

[Table of Contents](#)

[Description](#)

[Relevant Resources](#)

[Tutorial](#)

- [1. Install Duo 2-Factor application on your phone](#)
- [2. Download the Cisco Secure Client VPN software](#)
- [3. Launch the Cisco VPN client and connect to vpn.scu.edu](#)
- [4. SSH into the cloud instance](#)
- [5. Explore available directories](#)
- [6. Introducing SLURM](#)
- [7. Leveraging SLURM](#)
- [8. Nvidia SMI](#)
- [9. Getting up and running](#)
- [10. Saving and logging out](#)

[Closing Notes](#)

Description

This guide will provide step-by-step instructions on how you can connect to SCU's WAVE High Performance Computing (HPC) via your personal computer and run jobs with SLURM.

The WAVE HPC is a high performance server-platform that allows users to leverage high performance storage, GPUs, and large memory across high-bandwidth networking. It is hosted within the University's data center. According to SCU's website, there are two way to access the WAVE HPC:

- Local connection via the on-campus SCU network.
- Global connection via SCU VPN installed on your personal computer.

Given the greater flexibility of the latter, this guide will not cover the former.

Relevant Resources

Before we dive into the tutorial, it is my opinion that the six best resources for creating a VPN connection, using SLURM, and referencing Nvidia SMI are included below. Please have the tabs of them ready to complete each step.

WAVE HPC Overview	https://www.scu.edu/wave/wave-hpc/
Duo 2-Factor Authentication	https://www.scu.edu/technology/get-connected/duo/
How to Access SCU VPN	https://www.scu.edu/technology/get-connected/networking/how-to-access-vpn/
Wave HPC User Guide	https://wiki.wave.scu.edu/doku.php?id=hpc_userguide:hpc_access#secure_shell_access
SLURM Documentation	https://slurm.schedmd.com/documentation.html
Nvidia SMI	https://docs.nvidia.com/deploy/nvidia-smi/index.html

Tutorial

1. Install Duo 2-Factor application on your phone

In order to download the Cisco Secure Client VPN software, you will need to have the 2FA generator from DUO working on your phone. Refer to the DUO link above for full instructions.

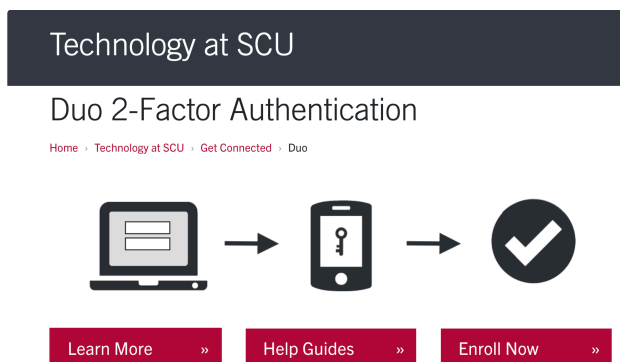


Figure I. SCU DUO website

Once installed and working, the generator should look like the following on your phone (for security purposes, I blocked out the code that was generated by example).

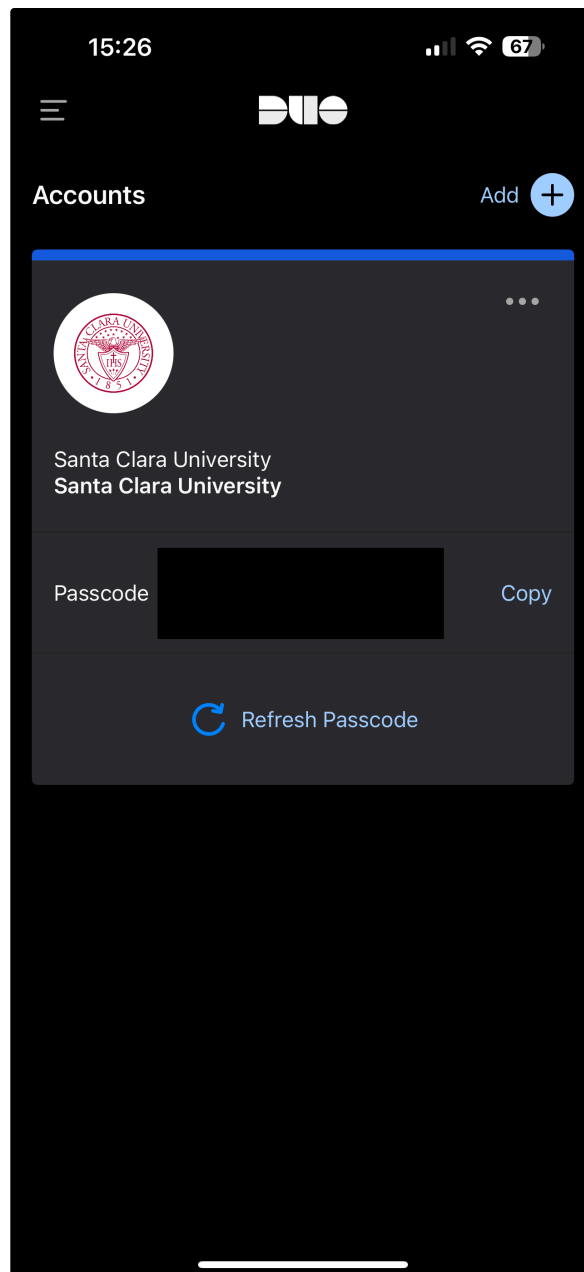


Figure II. Example DUO app

2. Download the Cisco Secure Client VPN software

Using the VPN link above to be redirect to the website vpn.scu.edu, use your credentials and 2FA app to download the Cisco VPN client. Once downloaded and opened on your respective operating system, follow the prompts to install the client, socket filter, and DART.

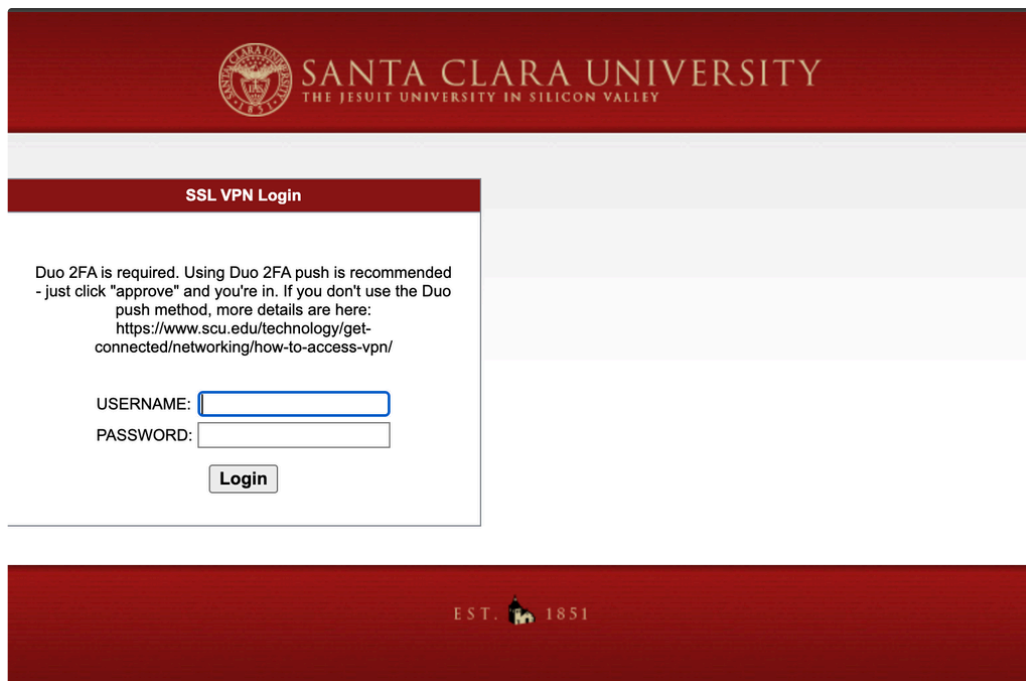


Figure III. SCU VPN website

3. Launch the Cisco VPN client and connect to vpn.scu.edu

Once the client and its dependencies have been installed, connect into the instance by entering `vpn.scu.edu` into the connection box. Once you press connect, you will be prompted to enter your standard SCU login credentials.

If you are having trouble, **first** check would be to make sure you entered your SCU password correctly. **Second** check would be to ensure that you aren't running any other VPNs in parallel which could conflict with the ports that the Cisco client might use.

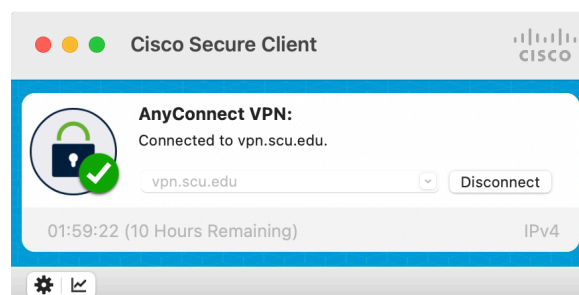


Figure IV. Cisco Secure Client on Mac

My example is running on Mac, but should be no different on Windows.

4. SSH into the cloud instance

Next, please refer to the link above for the WAVE HPC User Guide. Open up your operating system's terminal and **ssh** into the SCU cluster via the following command:

```
1 ssh <your SCU username>@login.wave.scu.edu
```

You might be asked about device finger printing - just say `yes`. When inside, you should notice that your user cursor looks different and attached via wave. Success!

```
maximiliankhan — mj Khan@wave8-login2:~ — ssh mj Khan@login.wave.scu.edu
(base) maximiliankhan@Maximilians-MacBook-Pro-3963 ~ % ssh mj Khan@login.wave.scu.edu
mj Khan@login.wave.scu.edu's password:
WARNING: The /WAVE/users and /WAVE/projects directories are backed up nightly.
Data in any other directories in WAVE are not backed up. Do not store
irreplaceable data in other directories without having a copy in one
of the directories that is backed up or somewhere externally. It is
not possible to recover any data lost or inadvertently deleted from
directories other than those listed above.
Last login: Sun Feb  2 13:41:43 2025 from 172.19.37.150
[mj Khan@wave8-login2 ~]$ whoami
mj Khan
[mj Khan@wave8-login2 ~]$ pwd
/WAVE/users2/unix/mj Khan
[mj Khan@wave8-login2 ~]$ ls
[mj Khan@wave8-login2 ~]$
```

Figure V. Example WAVE SSH connection

You will first connect into your personal unix folder (`/WAVE/users2/unix/<username>`). From here, we can traverse our available directory system and interact with various resources. To my surprise, `tmux` is also available for use at launch.

```
maximiliankhan — mj Khan@wave8-login2:/WAVE/scratch/CSEN-240-Wi25 — ssh mj Khan@login.w...
[mj Khan@wave8-login2 CSEN-240-Wi25]$
```

[0] 0: bash* "wave8-login2.wave.scu" 16:13 02-Feb-25

Figure VI. Example tmux execution

5. Explore available directories

Per the system administrator assigned to our class, we have access to the following external directories:

- `/WAVE/projects/CSEN-240-Wi25/`
- `/WAVE/scratch/CSEN-240-Wi25/`

To instantly move into any of previous directories, run the following, respective commands:

```
1 cd /WAVE/projects/CSEN-240-Wi25/
2 cd /WAVE/scratch/CSEN-240-Wi25/
```

I created a test files via `touch` under both our available class folders called `test.txt` using VIM (awesome that this is already installed on the compute node). To check them out, run `cat test.txt`.

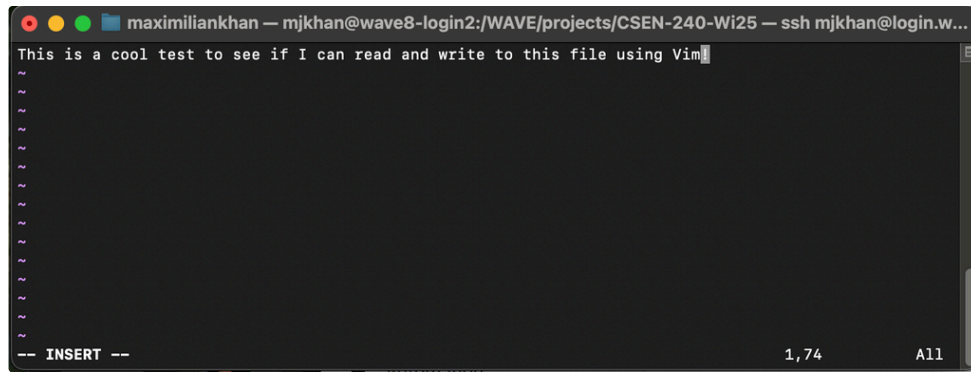


Figure VII. Using VIM inside the computer cluster

6. Introducing SLURM

SLURM is short for **Simple Linux Utility for Resource Management**. It is an open-source job scheduler typically used across HPC clusters, and IMO is similar to CRON or K9s. Using it helps ensure that all jobs are run efficiently against available CPU cores, memory, GPUs, and storage and enforces that resources and policies are fairly maintained. Our HPC system uses SLURM to schedule resources among all users since the system is a shared resource, and is required to run our jobs/programs.

Though the SCU WAVE wiki touches on this information, I'll give a quick overview on common SLURM commands:

sbatch	Submits a batch job script to the queue.
srun	Submits an interactive job or runs a command in parallel.
squeue	Views the job queue (see who is running what and which resources are used). <i>Will be useful for monitoring your scheduled job(s).</i>
scancel	Cancels a submitted job. <i>I would imagine that this would be very handy to remember in case you accidentally initiate something...</i>
sinfo	Displays information about available nodes and partitions.
scontrol show job <JOBID>	Shows detailed information your your running job. <i>Will be useful for monitoring your scheduled job(s).</i>

Here are example executions of `squeue`, `sinfo`, and `scontrol show job`.


```

maximiliankhan — mjkh@wave8-login2:/WAVE/scratch/CSEN-240-Wi25 — ssh mjkh@log...

[mjkh@wave8-login2 CSEN-240-Wi25]$ squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
48092 cmp Cr_test. mdeegan PD 0:00 1 (QOSMaxMemoryPerUser)
89966 cmp 100_svd_ gyellis R 14:54:32 1 mem02
89965 cmp 100_pca_ gyellis R 21:59:24 1 mem02
89961 cmp 100_pca_ gyellis R 1-04:56:20 1 mem01
90241 cpu parboil_ klor R 1:07:11 1 cpu02
90239 cpu plasmate klor R 1:24:42 1 cpu01
90206 gpu pccsptr0 zdong PD 0:00 1 (Dependency)
90248 gpu q3_zv zzhang13 R 7:41 1 gpu02
90225 gpu q3_zv zzhang13 R 12:26:42 1 gpu02
90205 gpu pcc0_nop zdong R 15:54:34 1 gpu04
89964 gpu 100_smi_ gyellis R 1-00:48:39 1 gpu04
89960 gpu 100_smi_ gyellis R 1-04:56:20 1 gpu03
89670 gpu pccsptr0 zdong R 1-22:11:46 1 gpu03
90235 hub iorais-n iorais R 2:23:54 1 hub01

[mjkh@wave8-login2 CSEN-240-Wi25]$ sinfo
PARTITION AVAIL TIMELIMIT NODES STATE NODELIST
all up 1-00:00:00 1 drain gpu01
all up 1-00:00:00 4 mix gpu[02-04],mem01
all up 1-00:00:00 3 alloc cpu[01-02],mem02
all up 1-00:00:00 2 idle cpu[03-04]
amd up 2-00:00:00 1 idle amd01
cmp* up 4-04:00:00 1 mix mem01
cmp* up 4-04:00:00 3 alloc cpu[01-02],mem02
cmp* up 4-04:00:00 2 idle cpu[03-04]
cpu up 4-04:00:00 2 alloc cpu[01-02]
cpu up 4-04:00:00 2 idle cpu[03-04]
gpu up 2-00:00:00 1 drain gpu01
gpu up 2-00:00:00 3 mix gpu[02-04]
hub up 1-00:00:00 2 down* hub[02,05]
hub up 1-00:00:00 1 mix hub01
hub up 1-00:00:00 6 idle hub[03-04,06-09]
mem up 4-04:00:00 1 mix mem01
mem up 4-04:00:00 1 alloc mem02
tst up 1-00:00:00 1 idle hub10
condo up 2-00:00:00 2 alloc rgrotjahn[01,04]
condo up 2-00:00:00 8 idle bio[01-03],dmlab01,rgrotjahn[02-03],ycho[201-202]

[mjkh@wave8-login2 CSEN-240-Wi25]$ scontrol show job 90248
JobId=90248 JobName=q3_zv
UserId=zzhang13(10220) GroupId=zzhang13(10220) MCS_label=N/A
Priority=1 Nice=0 Account=yliu_lab QOS=normal
JobState=RUNNING Reason=None Dependency=(null)
Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
RunTime=00:18:10 TimeLimit=2-00:00:00 TimeMin=N/A
SubmitTime=2025-02-02T16:13:00 EligibleTime=2025-02-02T16:13:00
AccrueTime=2025-02-02T16:13:00
StartTime=2025-02-02T16:13:00 EndTime=2025-02-04T16:13:00 Deadline=N/A
PreemptEligibleTime=2025-02-02T16:13:00 PreemptTime=None
SuspendTime=None SecsPreSuspend=0 LastSchedEval=2025-02-02T16:13:00 Scheduler=Backfill
Partition=gpu AllocNode:Sid=wave8-login2.wave.scu.edu:1822164
ReqNodeList=(null) ExcNodeList=(null)
NodeList=gpu02
BatchHost=gpu02
NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
ReqTRES=cpu=1,mem=30G,node=1,billing=1,gres/gpu=1
AllocTRES=cpu=2,mem=60G,node=1,billing=2,gres/gpu=1
Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
MinCPUsNode=1 MinMemoryCPU=30G MinTmpDiskNode=0
Features=(null) DelayBoot=00:00:00
OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
Command=/WAVE/users2/unix/zzhang13/dttr/xxxxx/condi_coarse_clip_txt_s2_version2.sh
WorkDir=/WAVE/users2/unix/zzhang13/dttr/xxxxx
StdErr=/WAVE/users2/unix/zzhang13/dttr/xxxxx/q3_zv-90248.out
StdIn=/dev/null
StdOut=/WAVE/users2/unix/zzhang13/dttr/xxxxx/q3_zv-90248.out
TresPerNode=gres/gpu:1
MailUser=zzhang13@scu.edu MailType=INVALID_DEPEND,BEGIN,END,FAIL,REQUEUE,STAGE_OUT

[mjkh@wave8-login2 CSEN-240-Wi25]$

```

Figure VIII. Example runs of squeue, sinfo, and scontrol show job

7. Leveraging SLURM

SLURM has the ability to leverage bash scripts to initiate jobs. They formally called **job scripts** or **batch scripts**. Similar to how you would add the **shebang** at the top of a bash file to tell the system which interpreter will execute the script, we use **SBATCH** at the top of bash files to tell SLURM what environment variables will be set for our job.

```
#!/bin/bash
#
#SBATCH --job-name=someapp
#SBATCH --output=somejob-%j.out
#
#SBATCH --partition=cmp
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=6
#SBATCH --mem-per-cpu=4G
#SBATCH --time=02:00:00
#
#SBATCH --mail-user=user_account@scu.edu
#SBATCH --mail-type=END
# Load the software to run the program
module load Python
# Run the Program
python ./somepythonprog -o outdir -a param1 -b param2 ...
```

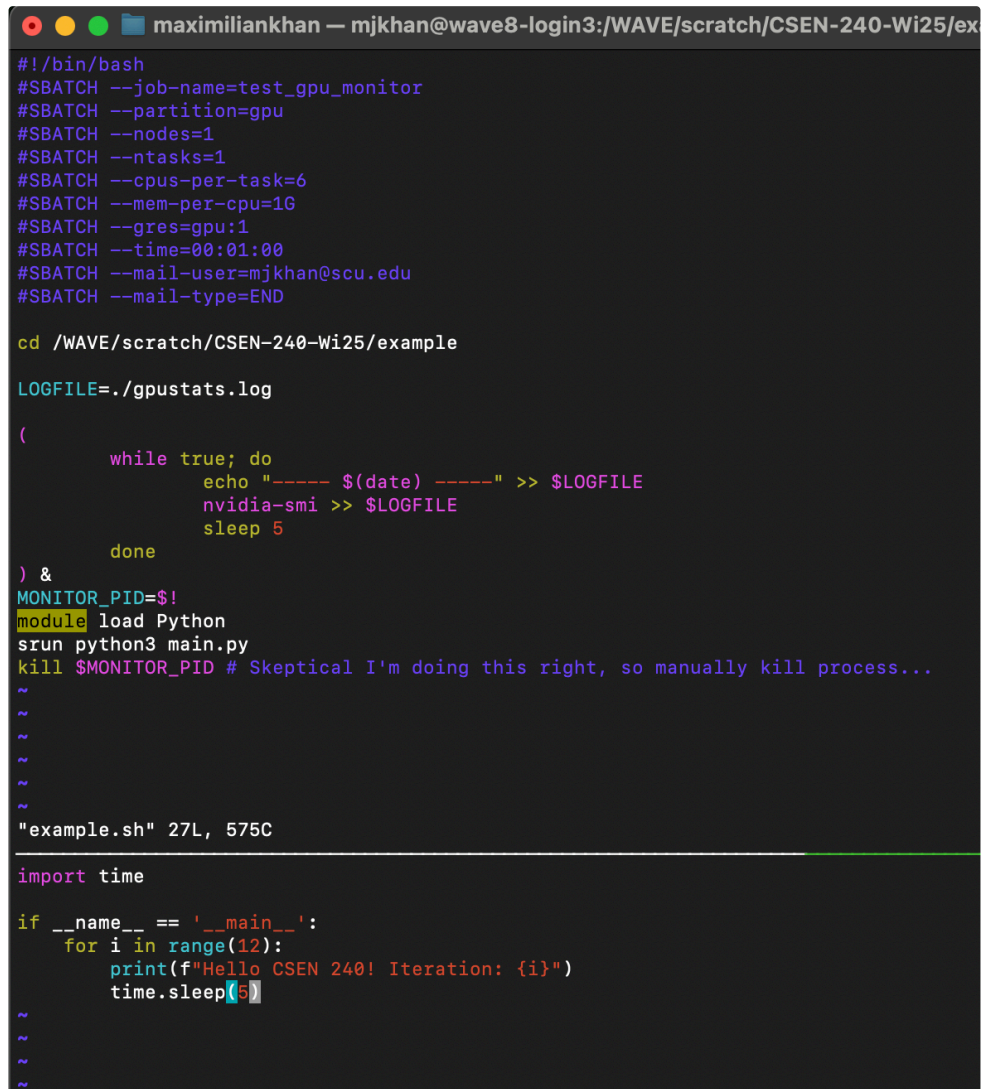
Figure IX. Example job script via SCU WAVE wiki

Here are some of the common SBATCH directives/variables used within a job script:

#SBATCH --job-name=<name>	Sets a human-friendly name for the job, and makes it easier to identify in the queue.
#SBATCH --output=<file>	Specifies the file where the standard output (stdout) of your job will be written. <i>Local relative to where this script is executed in the case that a full path is not provided.</i>
#SBATCH --error=<file>	Specifies the file where the standard error (stderr) of your job will be captured. <i>Local relative to where this script is executed in the case that a full path is not provided.</i>
#SBATCH --time=<hh:mm:ss>	Defines the maximum wall clock time that the job is allowed to run.
#SBATCH --partition=<partition_name>	Indicates which partition or job queue the job should run in. <i>I noticed that our SCU cluster is using the following partition variables as a pattern:</i> • cmp (unknown what this means) • cpu • gpu • hub (central compute point)
#SBATCH --nodes=<num_nodes>	Specifies the number of compute nodes required for the job.

9. Getting up and running

I wrote a quick job script (`example.sh`) and Python (`main.py`) program that can serve as an example template for setting up a job and allocating resources. You can find them under `/WAVE/scratch/CSEN-240-Wi25/example` . In the example below, the top box shows the bash file, and the bottom box shows the python code.



```
maximiliankhan — mj Khan@wave8-login3:/WAVE/scratch/CSEN-240-Wi25/ex
#!/bin/bash
#SBATCH --job-name=test_gpu_monitor
#SBATCH --partition=gpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=6
#SBATCH --mem-per-cpu=1G
#SBATCH --gres=gpu:1
#SBATCH --time=00:01:00
#SBATCH --mail-user=mj Khan@scu.edu
#SBATCH --mail-type=END

cd /WAVE/scratch/CSEN-240-Wi25/example

LOGFILE=./gpustats.log

(
    while true; do
        echo "----- $(date) -----" >> $LOGFILE
        nvidia-smi >> $LOGFILE
        sleep 5
    done
) &
MONITOR_PID=$!
module load Python
srun python3 main.py
kill $MONITOR_PID # Skeptical I'm doing this right, so manually kill process...
~
~
~
~
~
~
"example.sh" 27L, 575C

import time

if __name__ == '__main__':
    for i in range(12):
        print(f"Hello CSEN 240! Iteration: {i}")
        time.sleep(5)
~
~
~
```

Figure XI. Example code for job script and execution

After applying `chmod +x example.sh` against the job script, I ran it via `./example.sh` . In the example below, you can see the job allocated in `squeue` .

```

maximiliankhan — mjkh@wave8-login3:/WAVE/scratch/CSEN-240-Wi25/example — ssh
[mjkh@wave8-login3 example]$ ./example.sh
Hello CSEN 240! Iteration: 0
Hello CSEN 240! Iteration: 1
Hello CSEN 240! Iteration: 2
Hello CSEN 240! Iteration: 3
Hello CSEN 240! Iteration: 4
Hello CSEN 240! Iteration: 5
Hello CSEN 240! Iteration: 6
Hello CSEN 240! Iteration: 7
Hello CSEN 240! Iteration: 8
Hello CSEN 240! Iteration: 9
Hello CSEN 240! Iteration: 10
Hello CSEN 240! Iteration: 11
[mjkh@wave8-login3 example]$

[mjkh@wave8-login3 example]$ squeue

```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
48092	cmp	Cr_test.	mdeegan	PD	0:00	1	(QOSMaxMemoryPerUser)
90265	cmp	python3	mjkh	R	0:04	1	mem01
89965	cmp	100_pca_	gyellis	R	1-00:03:26	1	mem02
89961	cmp	100_pca_	gyellis	R	1-07:00:22	1	mem01
90239	cpu	plasmate	klor	R	3:28:44	1	cpu01
90206	gpu	pccsptr0	zdong	R	15:26	1	gpu03
90248	gpu	q3_zv	zzhang13	R	2:11:43	1	gpu02
90225	gpu	q3_zv	zzhang13	R	14:30:44	1	gpu02
90205	gpu	pcc0_nop	zdong	R	17:58:36	1	gpu04
89964	gpu	100_smi_	gyellis	R	1-02:52:41	1	gpu04
89960	gpu	100_smi_	gyellis	R	1-07:00:22	1	gpu03
90263	hub	iorais-n	iorais	R	6:47	1	hub01

```

[mjkh@wave8-login3 example]$ squeue

```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
48092	cmp	Cr_test.	mdeegan	PD	0:00	1	(QOSMaxMemoryPerUser)
90265	cmp	python3	mjkh	R	0:45	1	mem01
89965	cmp	100_pca_	gyellis	R	1-00:04:07	1	mem02
89961	cmp	100_pca_	gyellis	R	1-07:01:03	1	mem01
90239	cpu	plasmate	klor	R	3:29:25	1	cpu01
90206	gpu	pccsptr0	zdong	R	16:07	1	gpu03
90248	gpu	q3_zv	zzhang13	R	2:12:24	1	gpu02
90225	gpu	q3_zv	zzhang13	R	14:31:25	1	gpu02
90205	gpu	pcc0_nop	zdong	R	17:59:17	1	gpu04
89964	gpu	100_smi_	gyellis	R	1-02:53:22	1	gpu04
89960	gpu	100_smi_	gyellis	R	1-07:01:03	1	gpu03
90263	hub	iorais-n	iorais	R	7:28	1	hub01

```

[mjkh@wave8-login3 example]$

```

Figure XII. Executing of job script

Though this example script does not utilize a GPU, I was able to store the GPU state of the single GPU I allocated during the job into a log file. Below are some example results.

